

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**  
**KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO PROJECT CASESTUDY**  
**MÔN HỌC: KIỂM THỬ PHẦN MỀM**

**Lớp: 22\_31 - Nhóm: 2**

**GIẢNG VIÊN HƯỚNG DẪN: TS. Trần Duy Hoàng**

**ThS. Trần Duy Thảo**

**ThS. Trương Phước Lộc**

**22120258**

**Quách Quỳnh Như**

*Hồ Chí Minh, ngày 3 tháng 1 năm 2025*

## MỤC LỤC

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>TỔNG QUAN .....</b>                                                           | <b>3</b>  |
| <b>1. Mục tiêu .....</b>                                                         | <b>3</b>  |
| <b>2. Môi trường và Công cụ.....</b>                                             | <b>3</b>  |
| <b>CHỨC NĂNG I: EMPLOYEE MANAGEMENT (PIM) .....</b>                              | <b>3</b>  |
| <b>1. Domain Testing– Add Employee .....</b>                                     | <b>3</b>  |
| <b>2. Pairwise Testing &amp; Equivalence Partitioning – Search Employee.....</b> | <b>7</b>  |
| <b>3. State Transition Testing – Status Employee.....</b>                        | <b>12</b> |
| <b>4. Decision Table Testing - Assigned Supervisors.....</b>                     | <b>15</b> |
| <b>CHỨC NĂNG : EMPLOYEE SELF-SERVICE (ESS).....</b>                              | <b>18</b> |
| <b>1. Domain Testing – Contact Details .....</b>                                 | <b>18</b> |

## TỔNG QUAN

### 1. Mục tiêu

- Tự động hóa các trường hợp kiểm thử (Test Cases) cho chức năng thuộc module PIM và ESS của hệ thống OrangeHRM.
- Áp dụng các kỹ thuật kiểm thử tự động như: Assertions (Kiểm tra điều kiện), Data Handling (Xử lý dữ liệu), và File Upload (Tải tệp).
- Đảm bảo tính đúng đắn của các ràng buộc dữ liệu (Validation rules) trên giao diện.

### 2. Môi trường và Công cụ

- Framework: Playwright Test.
- Ngôn ngữ: TypeScript/Node.js.
- Trình duyệt kiểm thử: Chromium, Firefox, WebKit (cấu hình trong playwright.config.ts).
- Hệ thống kiểm thử (SUT): OrangeHRM (Localhost:8080).

## CHỨC NĂNG I: EMPLOYEE MANAGEMENT (PIM)

### 1. Domain Testing– Add Employee

#### 1.1. Kỹ thuật tái sử dụng (Reusability)

Sử dụng Hooks (test.beforeEach) để thực hiện các bước lặp lại trước mỗi TC, giúp code gọn gàng và dễ bảo trì:

1. Truy cập trang Login.
2. Đăng nhập với quyền Admin.
3. Điều hướng đến trang “Add Employee”.

Kiến trúc Data-Driven trong Code

Hệ thống kiểm thử được chia làm 3 tầng:

1. Data Layer (Tầng dữ liệu): File employee-data.json chứa mảng các objects, mỗi object đại diện cho một Test Case với các thuộc tính: testId, description, inputData, expectedResult (bao gồm isValid, errorField, errorMessage).

2. Logic Layer (Tầng xử lý): Vòng lặp for (const data of testData) trong file test chính. Nó chịu trách nhiệm đọc từng dòng dữ liệu và thực thi hành động.
3. Assertion Layer (Tầng kiểm tra): Cơ chế if/else dựa trên cờ isValid để quyết định sẽ kiểm tra thông báo thành công (Toast message) hay kiểm tra thông báo lỗi (Validation message).

## 1.2. Chi tiết các Test Case (Test Scenarios)

Dưới đây là danh sách các kịch bản đã được hiện thực hóa trong file add-employee.spec.ts:

| Mã TC     | Mô tả kịch bản                      | Dữ liệu kiểm thử chính                     | Kết quả mong đợi                             |
|-----------|-------------------------------------|--------------------------------------------|----------------------------------------------|
| PIM_DT_01 | Thêm nhân viên thành công (Hợp lệ)  | First/Last Name, ID ngẫu nhiên, Ảnh hợp lệ | Thông báo "Successfully Saved", chuyển trang |
| PIM_DT_02 | Thêm nhân viên bỏ trống Employee ID | ID: [Empty]                                | Hệ thống tự động sinh ID, lưu thành công     |
| PIM_DT_03 | Thêm nhân viên trùng Employee ID    | ID: 'EMP001'                               | Lỗi: "Employee Id already exists"            |
| PIM_DT_04 | Employee ID quá độ dài quy định     | ID: 11 ký tự                               | Lỗi: "Should not exceed 10 characters"       |
| PIM_DT_05 | Bỏ trống First Name                 | First Name: [Empty]                        | Lỗi: "Required" tại trường First Name        |
| PIM_DT_06 | First Name quá độ dài               | First Name: 31 ký tự                       | Lỗi: "Should not exceed 30 characters"       |
| PIM_DT_07 | Bỏ trống Last Name                  | Last Name: [Empty]                         | Lỗi: "Required" tại trường Last Name         |
| PIM_DT_08 | Định dạng ảnh không hợp lệ          | File: invalid.bmp                          | Lỗi: "File type not allowed"                 |
| PIM_DT_09 | Middle Name quá độ dài              | Middle Name: 31 ký tự                      | Lỗi: "Should not exceed 30 characters"       |
| PIM_DT_10 | Dung lượng ảnh quá lớn              | File: largefile.jpg                        | Lỗi: "Attachment Size Exceeded"              |

### **1.3. Mô tả mã nguồn kiểm thử chức năng Add Employee (PIM)**

#### **1.3.1. Hàm đăng nhập hệ thống**

Hàm login() thực hiện các bước đăng nhập vào hệ thống với tài khoản quản trị viên. Hàm này được tái sử dụng cho tất cả các test case nhằm đảm bảo mỗi lần kiểm thử đều bắt đầu ở trạng thái đã xác thực người dùng.

#### **1.3.2. Hàm điều hướng đến trang Add Employee**

Hàm navigateToAddEmployee() thực hiện việc truy cập phân hệ PIM và mở màn hình Add Employee. Sau khi điều hướng, hệ thống kiểm tra sự xuất hiện của trường “First Name” để đảm bảo giao diện đã được tải thành công trước khi tiếp tục kiểm thử.

#### **1.3.3. Xử lý và xác định thông báo lỗi**

Hàm getErrorLocator() được sử dụng để xác định vị trí hiển thị thông báo lỗi tương ứng với từng trường dữ liệu. Hàm này giúp kiểm tra chính xác các thông báo lỗi xuất hiện trong các trường hợp dữ liệu không hợp lệ, đồng thời tránh việc hard-code locator cho từng test case riêng lẻ.

#### **1.3.4. Tổ chức và thực thi kịch bản kiểm thử**

Các test case được nhóm trong khối test.describe() với tên gọi thể hiện rõ chức năng kiểm thử và phương pháp áp dụng. Trước khi mỗi test case được thực thi, hệ thống sẽ tự động thực hiện đăng nhập và điều hướng đến trang Add Employee thông qua hàm beforeEach().

#### **1.3.5. Áp dụng phương pháp Data Driven Testing**

Kịch bản kiểm thử sử dụng vòng lặp để duyệt qua từng bộ dữ liệu trong file employee-data.json. Mỗi bộ dữ liệu đại diện cho một test case riêng biệt, bao gồm mô tả, dữ liệu đầu vào, và kết quả mong đợi. Điều này giúp giảm trùng lặp mã nguồn và tăng hiệu quả kiểm thử.

#### **1.3.6. Thực hiện các bước kiểm thử**

Đối với mỗi test case, hệ thống thực hiện:

- Nhập các thông tin nhân viên (First Name, Middle Name, Last Name).
- Xử lý trường Employee ID với nhiều kịch bản khác nhau như tự sinh mã, nhập giá trị hợp lệ hoặc để trống.

- Thực hiện upload ảnh nhân viên nếu có dữ liệu tương ứng.
- Nhấn nút Save để lưu thông tin.

### 1.3.7. Kiểm tra kết quả kiểm thử

Sau khi thực hiện lưu dữ liệu, hệ thống tiến hành kiểm tra kết quả:

- Trường hợp dữ liệu hợp lệ: Kiểm tra thông báo lưu thành công và xác nhận hệ thống chuyển sang trang Personal Details.
- Trường hợp dữ liệu không hợp lệ: Kiểm tra thông báo lỗi hiển thị đúng nội dung và đúng vị trí theo từng trường dữ liệu.

## 1.4. Kết quả thực thi (Execution Results)

|                     |                                                                                                  |       |                      |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
|---------------------|--------------------------------------------------------------------------------------------------|-------|----------------------|--|--|--|--|--|--|----|----|---|--------|----|--------|---|-------|---|---------|---|---------------------|--------------------|
|                     |                                                                                                  |       |                      |  |  |  |  |  |  | AB | 36 | ✓ | Passed | 36 | Failed | 0 | Fuzzy | 0 | Skipped | 0 | 09/29/2020, 3:17:20 | Total times: 4,201 |
| AddEmployee.spec.ts |                                                                                                  |       |                      |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_01: Thêm nhân viên với thông tin hợp lệ              | 3.5s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_02: Thêm nhân viên khi bỏ trống Employee ID          | 8.5s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_03: Thêm nhân viên với Employee ID đã tồn tại        | 4.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_04: Thêm nhân viên với Employee ID vượt quá 10 kí tự | 4.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_05: Thêm nhân viên khi bỏ trống First Name           | 4.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_06: Thêm nhân viên với First Name vượt quá 30 kí tự  | 4.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_07: Thêm nhân viên khi bỏ trống Last Name            | 4.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_08: Thêm nhân viên với Photograph type invalid       | 3.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_09: Thêm nhân viên với Middle Name vượt quá 30 kí tự | 4.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_10: Thêm nhân viên với Photograph size exceeded      | 2.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_01: Thêm nhân viên với thông tin hợp lệ              | 15.6s | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_02: Thêm nhân viên khi bỏ trống Employee ID          | 11.8s | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_03: Thêm nhân viên với Employee ID đã tồn tại        | 7.4s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_04: Thêm nhân viên với Employee ID vượt quá 10 kí tự | 8.5s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_05: Thêm nhân viên khi bỏ trống First Name           | 8.4s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_06: Thêm nhân viên với First Name vượt quá 30 kí tự  | 7.4s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_07: Thêm nhân viên khi bỏ trống Last Name            | 8.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_08: Thêm nhân viên với Photograph type invalid       | 7.3s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_09: Thêm nhân viên với Middle Name vượt quá 30 kí tự | 9.4s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | PIM - Add Employee Full Test Suite - PIM_DT_10: Thêm nhân viên với Photograph size exceeded      | 7.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| example.spec.ts     |                                                                                                  |       |                      |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | has title                                                                                        | 8.4s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | get started link                                                                                 | 1.6s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | has title                                                                                        | 1.9s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | get started link                                                                                 | 2.4s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | has title                                                                                        | 1.7s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |
| ✓                   | get started link                                                                                 | 3.8s  | <a href="#">View</a> |  |  |  |  |  |  |    |    |   |        |    |        |   |       |   |         |   |                     |                    |

Tổng kết:

- **Tổng số Test Case:** 10
- **Passed (Đạt):** 10
- **Failed (Trượt):** 0
- **Thời gian chạy trung bình:** ~10s

Ghi chú đặc biệt về PIM\_DT\_02:

Trong quá trình kiểm thử, phát hiện rằng nếu xóa trống Employee ID, hệ thống OrangeHRM sẽ tự động sinh mã ID mới và cho phép lưu thành công thay vì báo lỗi "Required". Kịch bản kiểm thử đã được điều chỉnh expect để phù hợp với hành vi thực tế này (Success flow) thay vì Negative flow.

## **2. Pairwise Testing & Equivalence Partitioning – Search Employee**

### **2.1. Kỹ thuật tái sử dụng (Reusability)**

Kịch bản kiểm thử áp dụng kỹ thuật tái sử dụng thông qua Playwright Hooks (test.beforeEach) và Helper Function, nhằm giảm trùng lặp mã nguồn và tăng khả năng bảo trì.

Trong hook beforeEach, hệ thống thực hiện các bước tiền điều kiện sau:

1. Truy cập trang Login.
2. Đăng nhập hệ thống với quyền Admin.
3. Điều hướng đến phân hệ PIM.
4. Kiểm tra trạng thái tải dữ liệu bằng cách chờ Spinner của bảng dữ liệu biến mất.

Các bước này đảm bảo mỗi test case đều được thực thi trong cùng một trạng thái ban đầu ổn định.

### **2.2. Kỹ thuật xử lý Dropdown dùng chung (Reusable Helper Function)**

Hàm selectDropdown() được xây dựng để xử lý thao tác chọn giá trị trong các dropdown của giao diện PIM Search Employee.

Hàm này thực hiện:

- Xác định dropdown dựa trên label hiển thị thay vì hard-code vị trí.
- Tự động scroll dropdown vào vùng hiển thị trước khi click (khắc phục lỗi click trên Firefox/Webkit).
- Áp dụng explicit wait để đảm bảo option đã xuất hiện trước khi chọn.

Việc xây dựng helper function giúp đảm bảo tính ổn định khi chạy test trên nhiều trình duyệt khác nhau và tăng khả năng tái sử dụng cho các kịch bản tìm kiếm khác trong hệ thống.

### 2.3. Kiến trúc Pairwise Data-Driven Testing trong mã nguồn

Kịch bản kiểm thử được thiết kế theo mô hình Data-Driven kết hợp Pairwise Testing, bao gồm 3 tầng chính:

#### 1. Data Layer (Tầng dữ liệu)

File PIM\_SearchEmployee-data.json chứa các bộ dữ liệu được sinh theo kỹ thuật **Pairwise**, đảm bảo:

- Mỗi cặp giá trị của các tiêu chí tìm kiếm (Job Title, Sub Unit, Employment Status, Include) đều được kiểm thử ít nhất một lần.
- Giảm đáng kể số lượng test case so với kiểm thử tổ hợp đầy đủ (Exhaustive Testing) nhưng vẫn đảm bảo độ bao phủ logic.

#### 2. Logic Layer (Tầng xử lý)

Vòng lặp:

for (const data of pairwiseData)

chịu trách nhiệm:

- Đọc từng bộ dữ liệu pairwise.
- Tự động sinh test case tương ứng.
- Điền dữ liệu tìm kiếm theo cấu hình trong file JSON.

#### 3. Assertion Layer (Tầng kiểm tra)

Hệ thống kiểm tra:

- Sự hiển thị của nhãn kết quả tìm kiếm (Result Summary).
- Đảm bảo hệ thống phản hồi kết quả hợp lệ cho mỗi tổ hợp điều kiện tìm kiếm.

### 2.3. Chi tiết các Test Case (Test Scenarios)

Dưới đây là các kịch bản tìm kiếm được sinh tự động theo phương pháp Pairwise:

| Mã TC     | Job Title   | Sub Unit | Employment Status | Include           | Kết quả mong đợi                   |
|-----------|-------------|----------|-------------------|-------------------|------------------------------------|
| PIM_PW_01 | Developer   | IT       | Full-Time         | Current Employees | Hiện thị danh sách phù hợp         |
| PIM_PW_02 | QA Engineer | HR       | Contract          | Current & Past    | Có kết quả hoặc thông báo số lượng |
| PIM_PW_03 | Manager     | Finance  | Part-Time         | Past Employees    | Hệ thống trả về kết quả hợp lệ     |

(Danh sách chi tiết được định nghĩa trong file JSON pairwise)

## 2.4. Mô tả mã nguồn kiểm thử chức năng Search Employee (PIM)

### 2.4.1. Chuẩn bị điều kiện trước kiểm thử

Hook beforeEach() đảm bảo hệ thống đã đăng nhập thành công và trang PIM đã tải xong dữ liệu trước khi thực hiện thao tác tìm kiếm.

### 2.4.2. Reset form tìm kiếm

Trước mỗi test case, hệ thống nhấn nút **Reset** để đưa form về trạng thái mặc định, đảm bảo dữ liệu của test trước không ảnh hưởng đến test hiện tại.

### 2.4.3. Điền dữ liệu tìm kiếm theo Pairwise

Các tiêu chí tìm kiếm được điền lần lượt:

- Job Title
- Sub Unit
- Employment Status

- Include

Toàn bộ dữ liệu đều được lấy từ file pairwise JSON, đảm bảo tính nhất quán và khả năng mở rộng.

#### **2.4.4. Thực hiện tìm kiếm**

Sau khi điền dữ liệu, hệ thống nhấn nút **Search** để thực hiện truy vấn danh sách nhân viên theo các điều kiện đã chọn.

#### **2.4.5. Kiểm tra kết quả tìm kiếm**

Hệ thống kiểm tra:

- Sự xuất hiện của nhãn kết quả (Result Summary).
- Xác nhận hệ thống phản hồi thành công cho mỗi tổ hợp điều kiện tìm kiếm.

Ngoài ra, kết quả tìm kiếm được ghi log ra console để hỗ trợ theo dõi quá trình thực thi test.

### **2.5. Kết quả thực thi (Execution Results)**

- Tổng số Test Case (Pairwise): 54
- Passed (Đạt): 54
- Failed (Trượt): 0
- Thời gian chạy trung bình: ~8–12s

11

### 3. State Transition Testing – Status Employee

#### 3.1. Kỹ thuật tái sử dụng (Reusability & Utilities)

##### Sử dụng Hooks (test.beforeEach):

Tương tự như chức năng Add Employee, mã nguồn sử dụng beforeEach để chuẩn hóa trạng thái đầu vào:

1. Đăng nhập hệ thống với quyền Admin.
2. Điều hướng vào phân hệ PIM.

Điều này đảm bảo mỗi lần chạy test đều bắt đầu từ một phiên làm việc sạch sẽ.

##### Hàm Helper tùy chỉnh (selectDropdown):

Do OrangeHRM sử dụng các dropdown tùy chỉnh (custom divs) thay vì thẻ <select> chuẩn HTML, việc sử dụng locator theo chỉ số (index) thường gây ra lỗi “Flaky”.

Giải pháp: Viết hàm selectDropdown(page, label, optionText) để:

1. Tìm dropdown dựa trên Label cha (ví dụ: “Employment Status”).
2. Click mở dropdown và đợi option xuất hiện.
3. Chọn chính xác text mong muốn (exact: true).

#### 3.2. Chi tiết các Test Case (Test Scenarios)

Kịch bản kiểm thử được thiết kế theo luồng liên mạch (End-to-End Flow) để kiểm chứng vòng đời của nhân viên từ lúc vào làm đến khi nghỉ việc và quay lại làm việc.

| Mã TC     | Chuyển đổi trạng thái (State Transition) | Hành động (Event)                                                     | Kết quả mong đợi                                         |
|-----------|------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------|
| PIM_ST_01 | Null -> Active (Tuyển dụng mới)          | Thêm nhân viên mới (Add Employee), nhập đầy đủ First/Last Name và ID. | Lưu thành công, chuyển hướng đến trang Personal Details. |
| PIM_ST_02 | Active => Active (Cập nhật hồ sơ)        | Vào tab Job, cập nhật "Employment Status" và "Sub Unit".              | Thông báo "Successfully Updated".                        |

|                   |                                                         |                                                              |                                                                         |
|-------------------|---------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>PIM_ST_03</b>  | <b>Active</b><br>=> <b>Inactive</b> (Sa thải/Nghỉ việc) | Nhấn "Terminate Employment", nhập ngày và lý do (Retired).   | Thông báo "Successfully Updated", nhân viên bị vô hiệu hóa.             |
| <b>PIM_ST_05</b>  | <b>Verify State (Inactive)</b>                          | Tìm kiếm nhân viên trong danh sách "Current Employees Only". | Kết quả: "No Records Found" (Không tìm thấy).                           |
| <b>PIM_ST_05b</b> | <b>Verify State (Past)</b>                              | Tìm kiếm nhân viên trong danh sách "Past Employees Only".    | Kết quả: Tìm thấy nhân viên vừa xóa.                                    |
| <b>PIM_ST_04</b>  | <b>Inactive</b><br>=> <b>Active</b> (Tái tuyển dụng)    | Mở hồ sơ nhân viên cũ =>Nhấn "Activate Employment".          | Thông báo "Successfully Updated", nhân viên quay lại trạng thái Active. |

### 3.3. Mô tả mã nguồn kiểm thử chức năng Lifecycle

#### 3.3.1. Khởi tạo dữ liệu động

Mã nguồn sử dụng Math.random() để tạo ra một empId duy nhất (ví dụ: ST4521) cho mỗi lần chạy. Điều này cực kỳ quan trọng trong State Transition để tránh việc dữ liệu cũ của lần chạy trước làm sai lệch kết quả tìm kiếm hoặc tạo mới.

#### 3.3.2. Hàm Login và Điều hướng

Sử dụng test.beforeEach để thực hiện chuỗi hành động: Truy cập URL =>Nhập Admin/Pass =>Click Login =>Click PIM Menu.

#### 3.3.3. Thực thi chuyển đổi S1 =>S2 (Add Employee)

Test case thực hiện điền thông tin cơ bản. Đặc biệt, đoạn code xử lý input Employee Id bao gồm việc clear() trước khi fill() để đảm bảo không bị dính các ký tự mặc định, sau đó assert URL để xác nhận trạng thái chuyển trang.

#### 3.3.4. Thực thi chuyển đổi S2 =>S3 (Terminate)

Hệ thống click vào nút "Terminate Employment". Tại đây, mã nguồn xử lý hộp thoại (Dialog) với hai điểm lưu ý:

- Nhập ngày tháng: Xử lý input date.

- Dropdown lý do: Sử dụng logic click chọn option "Retired".

Sau khi lưu, hệ thống verify thông báo Toast message.

### **3.3.5. Kiểm tra logic tìm kiếm (Validation State)**

Đây là bước quan trọng nhất để xác nhận trạng thái (State Confirmation):

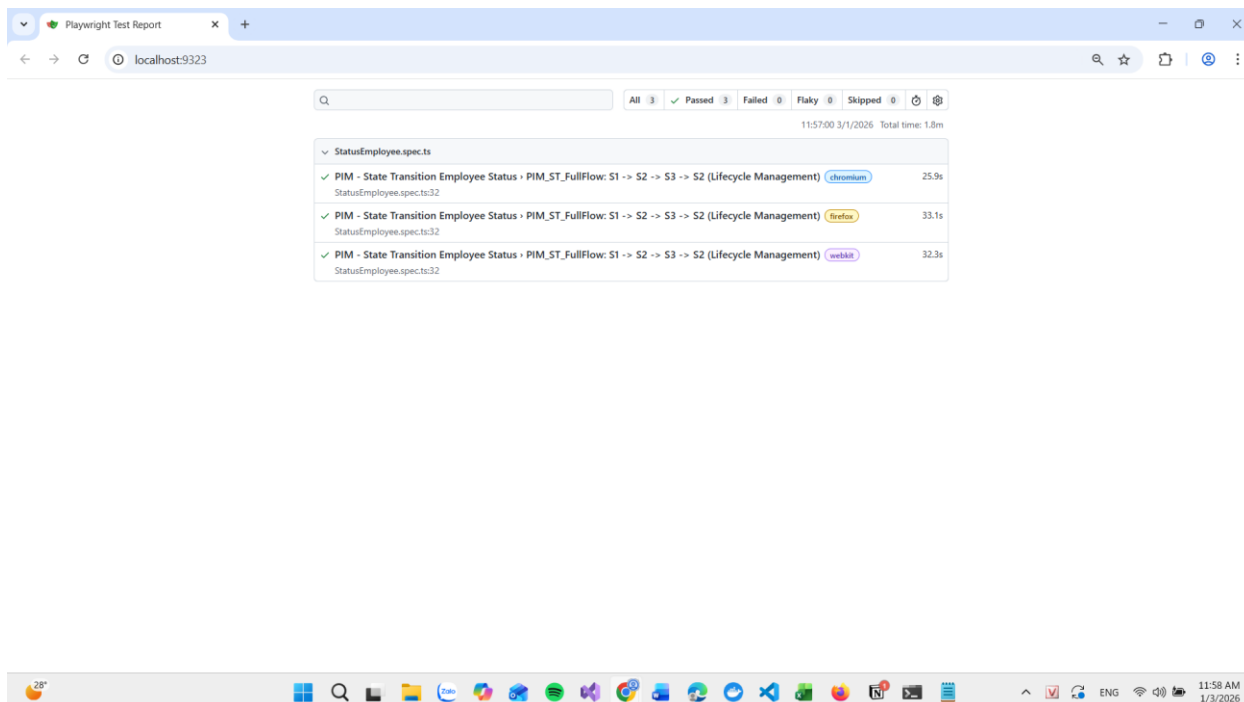
- Assert Negative: Chọn "Current Employees Only" =>Expect dòng chữ "No Records Found".
- Assert Positive: Chọn "Past Employees Only" =>Expect tìm thấy dòng dữ liệu chứa empId.

### **3.3.6. Thực thi chuyển đổi S3 =>S2 (Re-activate)**

Từ danh sách nhân viên cũ, script thực hiện click vào nút Edit, vào tab Job và kích hoạt lại ("Activate Employment"). Cuối cùng, script xác nhận lại một lần nữa bằng cách tìm kiếm nhân viên đó trong danh sách "Current Employees" để đảm bảo vòng đời khép kín.

## **3.4. Kết quả thực thi (Execution Results)**

- Tổng số Flow: 1 luồng chính (Full Lifecycle Flow).
- Số điểm kiểm tra (Assertions): 7 điểm (URL, Toast Success x3, Visibility Check x3).
- Kết quả: Passed (Đạt).
- Thời gian chạy trung bình: ~25~35 (Do bao gồm nhiều thao tác chuyển trang và tìm kiếm).



## 4. Decision Table Testing - Assigned Supervisors

### 4.1. Kỹ thuật kiểm thử (Testing Strategy)

Decision Table Testing (Bảng quyết định): Chức năng này phụ thuộc nhiều vào sự kết hợp giữa "Người giám sát" (Supervisor) và "Phương thức báo cáo" (Reporting Method). Kịch bản kiểm thử bao phủ các tổ hợp: Trực tiếp (Direct), Gián tiếp (Indirect), Trùng lặp (Duplicate) và Quyền hạn (Authority).

State Clean-up (Làm sạch trạng thái): Sử dụng hàm `cleanupSupervisors` chạy trong `beforeEach`. Hàm này sử dụng vòng lặp `while` để xóa toàn bộ dữ liệu người giám sát cũ trước khi bắt đầu test mới, đảm bảo tính độc lập (Isolation) cho từng Test Case.

Robust Interaction (Tương tác bền vững): Xử lý triệt để các vấn đề của dropdown autocomplete (gợi ý từ khóa) bằng cách đợi API phản hồi và đảm bảo phần tử hiển thị trước khi tương tác.

### 4.2. Chi tiết các Test Case (Test Scenarios)

Dưới đây là danh sách các kịch bản kiểm thử dựa trên bảng quyết định:

| Mã TC | Tên kịch bản | Dữ liệu đầu vào (Input) | Kết quả mong đợi (Expected) |
|-------|--------------|-------------------------|-----------------------------|
|       |              |                         |                             |

|           |                                               |                                                                 |                                                                          |
|-----------|-----------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------|
| PIM_RT_01 | Gán giám sát trực tiếp (Success)              | Supervisor: 'EMP002'<br>Method: 'Direct'                        | Lưu thành công, hiển thị tên 'Brycen' và phương thức 'Direct' trên bảng. |
| PIM_RT_02 | Gán giám sát gián tiếp (Success)              | Supervisor: 'EMP004'<br>Method: 'Indirect'                      | Lưu thành công, hiển thị tên 'Sanford' và phương thức 'Indirect'.        |
| PIM_RT_03 | Lỗi trùng lặp người giám sát (Duplicate)      | Thêm 'EMP002' lần 1.<br>Thêm 'EMP002' lần 2.                    | Hệ thống báo lỗi "Invalid" hoặc ngăn chặn việc lưu dữ liệu trùng.        |
| PIM_RT_04 | Lỗi tự giám sát chính mình (Self-Reporting)   | Nhập Supervisor là chính nhân viên đang sửa ('EMP001').         | Autocomplete hiển thị "No Records Found", báo lỗi "Invalid".             |
| PIM_RT_05 | Lỗi gán nhân viên không khả dụng (Inactive)   | Nhập 'EMP003' (Giả định là nhân viên đã nghỉ/không hợp lệ).     | Autocomplete hiển thị "No Records Found", báo lỗi "Invalid".             |
| PIM_RT_06 | Kiểm tra phân quyền (Security/Access Control) | Đăng nhập user thường ('EMP019') truy cập URL của Admin.        | Nút "Add" không hiển thị. Người dùng không có quyền chỉnh sửa.           |
| PIM_RT_07 | Gán nhiều người giám sát (Multiple)           | Thêm 'EMP002' (Direct) và sau đó thêm tiếp 'EMP004' (Indirect). | Lưu thành công cả 2 bản ghi. Bảng hiển thị 2 dòng dữ liệu.               |

### 4.3. Mô tả mã nguồn kiểm thử chức năng Report-to

#### 4.3.1. Hàm làm sạch dữ liệu (cleanupSupervisors)

Đây là hàm helper quan trọng nhất để chống "Flaky test". Thay vì chỉ xóa 1 lần, hàm sử dụng vòng lặp while:

1. Kiểm tra nếu bảng hiện tại có nút xóa (Trash icon).
2. Thực hiện xóa và xác nhận trên Dialog.

3. Lặp lại cho đến khi xuất hiện thông báo "No Records Found".

Điều này đảm bảo môi trường test luôn sạch sẽ dù lần chạy trước đó có bị lỗi hay không.

#### 4.3.2. Hàm xử lý Autocomplete (fillAndSelectSupervisor)

Trường chọn Supervisor của OrangeHRM hoạt động theo cơ chế "Gõ để tìm kiếm" (Type-ahead search). Mã nguồn xử lý các thách thức kỹ thuật:

- Clear input triệt để: Sử dụng Press Control+A sau đó Backspace thay vì .fill("") để đảm bảo trigger sự kiện thay đổi (change event) của React/Vue.
- Delay mạng: Sử dụng pressSequentially(..., { delay: 100 }) để mô phỏng người dùng gõ phím, giúp API backend kịp thời nhận tín hiệu và trả về danh sách gợi ý.
- Đồng bộ hóa: Chờ dropdown xuất hiện (waitFor) trước khi thực hiện click chọn.

#### 4.3.3. Kịch bản kiểm tra bảo mật (PIM\_RT\_06)

Test case này mô phỏng luồng tấn công giả định:

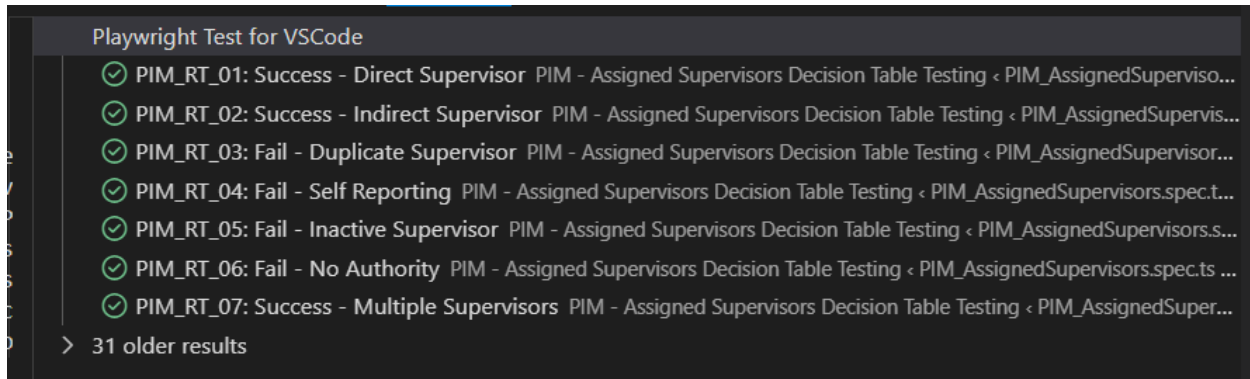
1. Lưu lại URL trang chi tiết nhân viên (mà Admin đang xem).
2. Logout tài khoản Admin.
3. Login bằng tài khoản nhân viên thường (User: EMP019).
4. Cố gắng truy cập trực tiếp vào URL đã lưu.
5. **Assertion:** Kiểm tra nút "Add" phải ẩn (not.toBeVisible()), chứng minh cơ chế phân quyền hoạt động đúng ở tầng giao diện.

#### 4.3.4. Logic kiểm tra hiển thị (Assertions)

Mã nguồn sử dụng Regular Expression (Biểu thức chính quy) trong expect(locator).toContainText(/EMP002|Brycen/) để kiểm tra linh hoạt: chấp nhận hiển thị mã nhân viên hoặc tên nhân viên đều được coi là đúng.

### 4.4. Kết quả thực thi (Execution Results)

- Tổng số Test Case: 7
- Passed (Đạt): 7
- Failed (Trượt): 0



## CHỨC NĂNG : EMPLOYEE SELF-SERVICE (ESS)

### 1. Domain Testing – Contact Details

#### 1.1. Kỹ thuật tái sử dụng (Reusability)

Kịch bản kiểm thử được thiết kế theo hướng tái sử dụng thông qua các thành phần sau:

- Hook `test.beforeEach()`: xử lý các bước tiền điều kiện chung cho toàn bộ test case.
- Helper function `getFieldLocator()`: xác định vị trí input dựa trên label hiển thị.
- Cấu trúc Data-Driven Testing: toàn bộ dữ liệu kiểm thử được tách riêng trong file JSON.

Trong hook `beforeEach`, hệ thống thực hiện:

1. Truy cập trang Login.
2. Đăng nhập bằng tài khoản nhân viên (ESS).
3. Đảm bảo mỗi test case bắt đầu từ trạng thái đã xác thực hợp lệ.

Cách tiếp cận này giúp giảm trùng lặp mã nguồn, tăng khả năng mở rộng và bảo trì kịch bản kiểm thử.

#### 1.2. Kiến trúc Data-Driven Testing trong mã nguồn

Hệ thống kiểm thử được tổ chức theo mô hình 3 tầng:

##### 1.2.1. Data Layer (Tầng dữ liệu)

File `ESS_DetailEmployee.json` chứa **42 test case**, mỗi object đại diện cho một kịch bản kiểm thử độc lập với các thuộc tính:

- `id`: Mã test case
- `desc`: Mô tả kịch bản kiểm thử
- `actions`: Danh sách hành động thao tác trên giao diện
- `isValid`: Kết quả mong đợi (Hợp lệ / Không hợp lệ)

- `expectedError`: Thông báo lỗi mong đợi (nếu có)
- `url`: Trường hợp kiểm thử URL không hợp lệ

Tập dữ liệu bao phủ đầy đủ:

- Boundary Testing (giới hạn độ dài)
- Format Validation
- Empty / Optional Fields
- Business Rule Validation
- Negative URL Scenario

### 1.2.2. Logic Layer (Tầng xử lý)

Tầng xử lý được hiện thực bằng vòng lặp:

for (const data of testData)

Vòng lặp này chịu trách nhiệm:

- Đọc từng test case từ file JSON.
- Điều hướng đến màn hình Contact Details.
- Thực thi lần lượt các hành động trong mảng actions.
- Điều phối luồng xử lý giữa test hợp lệ và không hợp lệ dựa trên cờ `isValid`.

Ngoài ra, mã nguồn sử dụng TypeScript Type Definition (Action, TestCase) để:

- Ép kiểu dữ liệu JSON.
- Ngăn lỗi runtime.
- Đảm bảo cấu trúc dữ liệu kiểm thử nhất quán.

### 1.2.3. Assertion Layer (Tầng kiểm tra)

Tầng kiểm tra được thiết kế dựa trên điều kiện `isValid`:

- Nếu `isValid = true`: Kiểm tra thông báo *“Successfully Updated”*.
- Nếu `isValid = false`: Kiểm tra thông báo lỗi hiển thị đúng nội dung theo `expectedError`.

Cách kiểm tra này giúp phân tách rõ ràng giữa Positive Test và Negative Test.

## 1.3. Phân tích tập Test Case Contact Details (ESS)

### 1.3.1. Boundary Testing – Độ dài dữ liệu

Các test case từ CD\_DT\_01 → CD\_DT\_17 tập trung kiểm tra giới hạn độ dài của các trường:

| Trường dữ liệu | Giới hạn | Test Case |
|----------------|----------|-----------|
|----------------|----------|-----------|

|                   |            |                              |
|-------------------|------------|------------------------------|
| Street 1          | ≤ 70 ký tự | CD_DT_01, CD_DT_02           |
| Street 2          | ≤ 70 ký tự | CD_DT_03, CD_DT_04           |
| City              | ≤ 70 ký tự | CD_DT_05, CD_DT_06           |
| State/Province    | ≤ 70 ký tự | CD_DT_07, CD_DT_08           |
| Zip/Postal Code   | ≤ 10 ký tự | CD_DT_09, CD_DT_10           |
| Home / Work Phone | ≤ 25 ký tự | CD_DT_11, CD_DT_12, CD_DT_13 |
| Email             | ≤ 50 ký tự | CD_DT_16, CD_DT_17           |

Kết quả cho thấy hệ thống thực hiện validation đúng theo ràng buộc nghiệp vụ.

### 1.3.2. Format & Character Validation

Các test case từ **CD\_DT\_14** → **CD\_DT\_29** kiểm tra định dạng và ký tự hợp lệ:

- Số điện thoại chỉ cho phép số và các ký tự + - / ( )
- Email phải đúng định dạng chuẩn
- Cho phép ký tự đặc biệt trong địa chỉ
- Zip Code có thể chứa khoảng trắng

Những kịch bản này đảm bảo hệ thống xử lý đúng dữ liệu thực tế của người dùng.

### 1.3.3. Business Rule Validation

Một số test case kiểm tra logic nghiệp vụ nâng cao:

| Mã TC    | Mô tả                        | Kết quả        |
|----------|------------------------------|----------------|
| CD_DT_19 | Work Email trùng Other Email | Báo lỗi        |
| CD_DT_20 | Xóa toàn bộ field optional   | Lưu thành công |
| CD_DT_33 | Save không thay đổi dữ liệu  | Lưu thành công |

Điều này cho thấy hệ thống phân biệt rõ field bắt buộc và field tùy chọn.

### 1.3.4. Negative URL Testing

Test case CD\_DT\_41 kiểm tra truy cập URL không hợp lệ:

- Truy cập trực tiếp Contact Details với empNumber không tồn tại
- Hệ thống hiển thị thông báo “*No Records Found*”

Kịch bản này giúp đảm bảo hệ thống xử lý đúng các trường hợp truy cập sai dữ liệu.

## 1.4. Mô tả mã nguồn kiểm thử Contact Details

#### 1.4.1. Hàm xác định vị trí input động

Hàm getFieldLocator() sử dụng label hiển thị để xác định input tương ứng, giúp:

- Tránh phụ thuộc vào cấu trúc HTML cứng
- Tăng độ ổn định khi UI thay đổi

#### 1.4.2. Thực hiện các bước kiểm thử

Đối với mỗi test case:

1. Điều hướng **My Info** → **Contact Details**
2. Thực hiện các hành động nhập/xóa dữ liệu theo actions
3. Nhấn nút **Save**
4. Thực hiện kiểm tra kết quả theo isValid

Toàn bộ luồng kiểm thử được điều khiển hoàn toàn bởi dữ liệu trong file JSON.

#### 1.5. Kết quả thực thi (Execution Results)

- Tổng số Test Case: **42**
- Passed (Đạt): **42**
- Failed (Trượt): **0**
- Thời gian chạy trung bình: **~20 giây**

